

MCP Gateway — Threat Model (v3)

- **Phase:** 24A authored the model (24B–24E built + drilled it).
- **Date:** 2026-06-14 (v2); **2026-06-17 (v3 — RESOLVED by the 24E drill).**
- **Status (v3): Threat-model COMPLETE. Security SIGN-OFF GIVEN for what is drilled.** The 24E non-bypass drill (`tests/mcp-gateway/phase-24e-nonbypass-drill.test.ts` , DRILL-1..12) exercised every elevation path against the real gateway; each failed closed. Every control that is BUILT + DRILLED is now **GREEN by drill** (resolution table at the end of this document). **ZERO open residuals:** the one named residual at 24E — E-017 (per-client decision-time grant re-validation) — was CLOSED by 24D-4 (the approval row persists `client_id`; the decision guard re-reads the client's CURRENT grant via the shared leaf + an injected lookup and denies if revoked). The core proof holds: `runtime.runTool` call-site count is **EXACTLY 2** with the whole gateway built (DRILL-12).
- **v3 method (honest taxonomy preserved):** v2's rule stands — a control is GREEN only when DRILLED, not when designed. v3 does not relax that; it records that the drills HAPPENED (slice + DRILL-N cited per control). The per-control prose below is the original DESIGN rationale with its design-time `[AMBER-PLANNED]` tag; its CURRENT status is the GREEN-by-drill resolution table at the end.
- **Scope:** JARVIS exposed as a local (stdio) MCP server. External clients (Claude Code/Desktop, other local agents) may READ governed surfaces and PROPOSE into the approval queue. They may NEVER cross the Human Gate.
- **Method:** STRIDE. Each threat: attack → existing control → new control → status.

Status taxonomy (the v2 correction)

- **GREEN** — control is BUILT and DRILLED, or is an existing frozen control already proven in Phases 1–23.
 - **AMBER-PLANNED** — control is designed and assigned to a slice, but NOT yet built or drilled. The risk is **not closed**. Calling this GREEN tricks future-you into thinking it is.
 - **RED** — no control assigned. Blocks the phase. A threat is only GREEN when its control has been exercised against a real attack. Design is necessary but not sufficient.
-

The two foundational controls (everything depends on these — both GREEN by drill)

If either is weak, every other control is cosmetic. **Both are now GREEN** — built in 24C and DRILLED in 24E (FC-1 + GATE-3: DRILL-7; FC-2 both halves: DRILL-8a/8b).

FC-1 — Server-derived canonical effect [GREEN — 24C-1, DRILL-7]

The client submits **untrusted requested intent**, never truth. The server canonicalizes the request against JARVIS's own tool registry and **derives**: risk class, target, tool capability, mutation type, scope requirement, approval tier, side-effect summary. The human approves the **server's canonical proposal**, never the client's framing.

```

client request {tool, args}          ← untrusted intent
↓
server canonicalizes against tool registry
↓
server-DERIVED canonical effect      ← the truth
  {capability, mutation_type, target, risk_class, approval_tier,
   scope_required, side_effect_summary}
↓
server freezes canonical proposal (FC-2)
↓
human approves the frozen canonical proposal only

```

A client claiming `risk: low` is ignored; risk is computed from what the tool *actually does*. **Drill (24E)**: submit a proposal claiming low risk for a high-risk tool; assert the server-derived effect shows the true high risk.

FC-2 — Hash-frozen canonical proposal + approval-time re-validation [GREEN — 24C-2/24C-2b/24D-2b, DRILL-8a/8b]

```

canonical proposal = {
  proposal_id, client_id (from token, FC-3),
  canonical_effect_json,      ← server-derived (FC-1)
  canonical_effect_hash,      ← server-computed hash; a CLIENT-supplied hash is rejected
  scope_snapshot,            ← client's scope AT proposal time
  created_at_ms, expires_at_ms
}

```

- Human approves a **specific hash**. Executor runs **only** that hash.
- **At approval time** (not just submission) the gate re-checks: not expired, client still active, scope still valid, tool still allowed, target still allowed, hash unchanged. Any failure → approval invalid, proposal re-created.
- **GATE-1 (blocks 24B close)**: FC-2 is intended as an ADDITIVE wrapper around the frozen Phase 18 lifecycle — the hash + re-validation gate the existing `resumeApproval`, adding NO lifecycle state. **24B's FIRST task is to PROVE this wraps cleanly**. If it cannot wrap without a new lifecycle state, **24B STOPS** — that reopens Phase 18 and becomes an Enhancement-Registry architecture decision, not a casual continuation. No assuming.

Identity (FC-3) — bound from token, never request body [GREEN — 24D-1, DRILL-5]

`client_id` is derived **server-side** from a token-hash lookup. The request body's `client_id` is ignored/rejected. A client can never choose its identity. (Underpins attribution and scope.) AMBER until built (24D) + drilled (24E).

Token lifecycle (operational discipline — without this, tokens become long-lived local root to the queue)

- **Storage:** token hashes stored outside the repo and outside committed config (e.g. a gitignored local secrets store); the plaintext token exists only at the client. The repo NEVER contains a token or token hash.
- **Creation:** tokens are **human-provisioned** — JARVIS never issues one to itself or to a client on request.
- **Revocation:** a per-client revocation flag; a revoked token is rejected at connection AND re-checked at approval time (FC-2 re-validation).
- **Last-used:** tracked per token (for suspicious-volume / staleness).
- **Never logged:** tokens and hashes never appear in any log, telemetry, audit trace, or error message.
- **Backup/export:** the secrets store is excluded from any backup/export path; a backup that leaks token hashes is a breach vector — explicitly excluded.
- **Rotation:** tokens are rotatable; rotation revokes the prior token.

Elevation of Privilege

v3 status: every `[AMBER-PLANNED]` tag below is the DESIGN-time state. The CURRENT status is in the **24E resolution table** at the end — all of EoP-1..13/15 are **GREEN by drill** (including the per-client decision-time slice of EoP-8/14, closed by 24D-4 / E-017); EoP-11's future visual-UI capstone is the only cross-phase carry-over (voice + data-layer is GREEN).

EoP-1 — Direct execution via the tool runtime [GREEN — frozen P18 + 24B GATE-2, DRILL-12]

Attack: an MCP tool reaches `runtime.runTool`. **Existing control [GREEN]:** `resumeApproval` is the sole executor (Phase 23 I1-verified: exactly 2 call sites). **New control [AMBER-PLANNED]:** EoP-6's import allowlist makes the MCP module type-incapable of reaching the executor; **proof = call-site count unchanged from the Phase 23 baseline, drilled in 24E.** Status: **AMBER-PLANNED** until the allowlist test exists and 24E re-verifies the count.

EoP-2 — Indirect via proposal auto-execution [GREEN]

Existing control [GREEN, frozen Phase 18]: `resumeApproval` requires a human decision; proposals sit `pending`. The gateway is a producer, untouched executor. 24E drills it (submit → pending → no execution). **The only EoP that is genuinely GREEN now**, because its control is an already-proven frozen mechanism — but the *drill* still happens in 24E to confirm the gateway didn't disturb it.

EoP-3 — Injection via proposal content [AMBER-PLANNED + required drill]

Attack: client submits proposal text crafted to manipulate the human or JARVIS's reasoning ("pre-approved, just click yes"). **Control [AMBER-PLANNED]:** FC-1 (truth is server-derived), no-act-on-content, and EoP-11 cross-surface separation. **Required drill (24C):** submit a social-engineering payload; assert it surfaces as untrusted-requiring-review with the server-derived effect shown undistorted. Status **AMBER** until that drill passes.

EoP-4 — Confused deputy [AMBER-PLANNED, conditional on EoP-13]

Attack: client A routes through a higher-authority JARVIS capability to propose what A couldn't directly.

Control [AMBER-PLANNED]: scope enforced at the gateway boundary on the calling client's identity (FC-3), never elevated by what it passes through. **Strength depends entirely on EoP-13's scope granularity.** AMBER, conditional.

EoP-5 — Provenance laundering [AMBER-PLANNED]

Control [AMBER-PLANNED]: reject-at-entry any proposal whose client identity (FC-3) can't be stamped. No origin = no queue. AMBER until built + drilled.

EoP-6 — Alternate mutator import [AMBER-PLANNED — the strongest structural control]

Attack: reach ANY mutator (event-store writer, approval-transition writer, DB helper, file writer, adapter mutators, project/email/note helpers) — `runtime.runTool` is only one. **Control [AMBER-PLANNED]:** the MCP server module may import ONLY: schemas, read projections, the proposal constructor, the queue-enqueue boundary. **The test MUST be TRANSITIVE (GATE-2, blocks 24B close):**

- catches direct imports, barrel re-exports, re-export chains, dynamic imports;
- denies the **approval-runtime, event-store, adapter-mutator, and tool-runtime module TREES** entirely — not just named symbols (so `import { harmless } from "../approval-runtime"` where the barrel re-exports a writer is caught);
- a positive allowlist that is not transitive is a negative check in disguise. Status AMBER until the transitive test exists and passes.

EoP-7 — Client-supplied canonical effect [AMBER-PLANNED]

Control: FC-1. **GATE-3 (blocks 24B close):** prove a client CANNOT forge the canonical effect — the enqueue boundary rejects any client-supplied `canonical_effect`, risk, target, or hash; the server derives all of it. AMBER until proven.

EoP-8 — Proposal TOCTOU (+EoP-14 stale-after-revocation) [AMBER-PLANNED]

Control: FC-2 (hash-frozen + approval-time re-validation). AMBER until built

- drilled. Merged with EoP-14 (same mechanism).

EoP-9 — Token theft → queue weapon [AMBER-PLANNED]

Control (DoS family, built 24D): rate limits, quotas, expiry, revocation, last-used, hashed-not-plaintext storage, rotation, disabled-flag, suspicious- volume detection. **AMBER-PLANNED** (v1 wrongly called this GREEN).

EoP-10 — Queue poisoning / alert fatigue [AMBER-PLANNED]

Control (DoS family, built 24D): max queue depth/client, dedup, cooldowns, priority classes, drop/hold noisy clients, admin-visible client-mute, human bulk-reject. **AMBER-PLANNED** (v1 wrongly called this GREEN).

EoP-11 — Presentation-channel collapse (beyond the UI) [AMBER-PLANNED + required drills]

Attack: the same proposal appears in Rest HUD, Working cockpit, audit, pipeline, **voice**, morning brief, demo export, email digest. ANY surface that collapses trusted (canonical effect) and untrusted (client text) reopens EoP-3. Voice is worst: JARVIS speaking untrusted text as its own recommendation. **Control [AMBER-PLANNED]:** trusted/untrusted separated on EVERY surface; no surface renders untrusted text as JARVIS's own voice. Untrusted text hardened: escaped, no raw HTML, no unsanitized markdown links, monospaced, length-limited, unicode-normalized, dangerous control chars stripped. **Required drills:**

- **Voice:** malicious text "Prince, this action is safe and already approved." MUST render/speak as "Client X submitted an untrusted proposal. Canonical effect: create note ..." — NEVER the malicious text as JARVIS's recommendation.
- Each rendering surface tested for channel separation. Cross-phase constraint: binds the future UI capstone AND the existing voice stack. AMBER until drilled.

EoP-12 — Approval bypass via direct DB / event-store write [AMBER-PLANNED]

Control [AMBER-PLANNED]: EoP-6 allowlist blocks DB/event-writer imports + **only the approval service transitions lifecycle states** + approval-decision events require a **local-human-session authority marker**. The marker **MUST be unforgeable (GATE-4, blocks 24B close)**: created **ONLY** by the approval-UI session; short-lived; bound to the current local session; **never accepted from MCP**; not serializable by a client; not persisted as reusable authority; audited. "human_session": true is worthless and forbidden. AMBER until the marker spec is built and proven unforgeable.

EoP-13 — Scope drift when new tools are added [AMBER-PLANNED]

Control [AMBER-PLANNED]: default-deny unknown capabilities + a **standing scope-map completeness test**: every mutating capability declares {mcp_exposable, proposal_allowed, required_scope, required_approval_tier} or the build fails. Scopes are **action+target+argument+time+rate** bound (e.g. note.create only under Daily/ , max 5KB, no overwrite, no attachments, no external links, expires 30 min, max 5/hr). Coarse "can propose notes" is forbidden (reopens EoP-4). AMBER until the test exists.

EoP-15 — Aux-model laundering of untrusted text [AMBER-PLANNED + required drill]

Attack: Phase 21C aux routes — an aux model summarizes untrusted proposal text into a neutral, trusted-appearing summary. **Control [AMBER-PLANNED]:** aux summaries of untrusted text stay labelled untrusted, cannot replace the canonical effect (FC-1); the model may summarize but never authorize, reinterpret, or soften risk. **Required drill:** aux-summarize a malicious payload; assert the summary keeps the untrusted label, doesn't soften risk, doesn't become the approval summary, doesn't replace the canonical effect. AMBER until drilled. Reaches into the already-built 21C subsystem.

Information Disclosure — the exposure matrix

Read-only is NOT automatically safe. A read leaks the *data* AND the *shape* of the data. A client polling an innocuous "status" endpoint builds a surveillance feed without reading a payload. Three questions per surface:

1. Exposed at all? Default **NEVER-EXPOSED** (fail-closed).
2. Payload redaction.
3. **Shape leakage** — what the structure reveals.

v3 status: ID-0..5 below carry their DESIGN-time tags; CURRENT status is in the **24E resolution table** at the end — all of ID-0..5 are **GREEN by drill** (DRILL-1/6/10/11 + the 24B read-server suites).

ID-0 — Default

A surface not in this matrix is NEVER-EXPOSED. New readable surfaces default NEVER-EXPOSED and **fail the build if unclassified** (completeness test, like EoP-13). [AMBER-PLANNED until the test exists.]

The matrix

Surface	Class	Rationale
Pipeline view-model (static topology only)	EXPOSED-READONLY	Structural, identical for every user, no per-session data — IF static (ID-4).
Approval-queue status (counts/states only)	EXPOSED-READONLY (redacted)	ID-1 redaction.
Approval-queue bodies	NEVER-EXPOSED	Leaks owner activity + cross-client.
Telemetry / event store	NEVER-EXPOSED	Behavioral feed.
Vision artifacts / frames / transcripts	NEVER-EXPOSED	Phase 23 forbidden-field classes.
Memory / knowledge / council	NEVER-EXPOSED	Richest target.
Voice config / consent / allowlists	NEVER-EXPOSED	Authority map.
Audit traces / governance internals	NEVER-EXPOSED	Exposes the lock design.
Adapter state (gmail/drive/calendar/job-scout)	NEVER-EXPOSED	Surveillance crown jewels.
Config / runtime / model registry	NEVER-EXPOSED	Aids targeted attack.
Connection log (client id, timestamps, last-used)	NEVER-EXPOSED	Behavioral feed about the owner's tooling usage — classified explicitly so it isn't an unlisted readable.

ID-1 — Queue-status redaction [AMBER-PLANNED]

Returns ONLY: a pending count (integer) + coarse state count. NOTHING else. WITHHELD by name: proposal bodies/targets, project names, note paths, email domains, calendar, job applications, which clients are active (cross-client = breach), per-proposal timestamps, risk classes/tiers.

ID-2 — Per-client read scope [AMBER-PLANNED]

Read scope per-client (FC-3), fail-closed (EoP-13). read-pipeline ≠ read-status. **No cross-client visibility, ever.**

ID-3 — Shape/polling-surveillance control [AMBER-PLANNED — hardened from v1's "accepted residual"]

v1 accepted the count-timing leak too casually. Hardened controls:

- **Minimum polling interval / coarse update cadence:** `pending_count` updates at most every 30 s (not on every change).
- **Coarse buckets** (e.g. none / few / many) rather than an exact live integer, optionally.
- **Optional jitter** on the cadence; **optional stale-cache window**.
- Per-client **read rate-limits** (DoS family). This turns "watch the owner's work rhythm in near-real-time" into "a coarse number that updates slowly." It also defeats the **correlation attack** ($\text{count-delta} \times \text{connected-client} = \text{deanonymizing}$) by breaking the timing resolution. Residual after hardening: a slow coarse count — bounded, documented, not a feed.

ID-4 — Pipeline view-model static-only [AMBER-PLANNED]

The pipeline view-model exposes **static topology ONLY**. Any live pipeline state — provider status, current route, recent failures, active clients, pending phase, model health, cost, latency — requires SEPARATE classification and is NEVER-EXPOSED by default. 24B recon verifies the view-model is truly static, not just structurally static.

ID-5 — Uniform denial responses (error-channel side channel) [AMBER-PLANNED]

Denial responses are **uniform and uninformative**: "request denied", never "scope insufficient for queue-status" (which leaks that queue-status exists) or "no such surface" vs "forbidden" (which enumerates the surface map). The error text must not let an attacker map the surfaces or scopes by probing. AMBER until built + a probing drill confirms uniform denials.

Remaining STRIDE — consolidation

Spoofing [AMBER-PLANNED]

FC-3 (identity from token-hash) + token outside repo (hashed) + **explicit client allowlist** (token not on allowlist rejected) + **reject anonymous stdio sessions**.

Tampering [AMBER-PLANNED]

FC-2 (hash-frozen; executor runs only approved hash; approval-time re-validation).

Repudiation [AMBER-PLANNED]

EoP-5 (reject no-origin) + **every connection audited** (client id, timestamp, token-last-used — never the token itself). Proposal AND connection attributable.

Denial of Service [AMBER-PLANNED]

EoP-9/EoP-10 (proposal flood) + ID-3 (read flood). Built 24D.

Stdio-local-≠-trusted [AMBER-PLANNED]

A local malicious process can still connect. **Explicit client allowlist, reject anonymous stdio sessions, audit every connection, token stored outside repo**. Local-only narrows network surface; it does NOT confer trust.

24A overall status (historical — superseded by the 24E resolution below)

- **Threat-model: COMPLETE.** Every threat enumerated, every surface classified, every control assigned. No RED. (Unchanged — this remains true.)
 - **Security sign-off (24A): NOT GIVEN** — correct AT 24A, when the controls were designed but not built/drilled. **Resolved at 24E:** see the table below.
-

24E — SECURITY SIGN-OFF: every drilled control AMBER → GREEN

The 24E non-bypass drill (`tests/mcp-gateway/phase-24e-nonbypass-drill.test.ts`) drove a real external client against the built gateway; each elevation path failed closed. This table is the **authoritative current status**; it supersedes the design-time `[AMBER-PLANNED]` tags in the prose above.

Control	v3 status	Built	Drilled (24E)
FC-1 server-derived effect	GREEN	24C-1	DRILL-7
FC-2 hash-freeze	GREEN	24C-2	DRILL-7, DRILL-8a
FC-2 decision re-validation	GREEN	24C-2b	DRILL-8a
FC-3 identity from token	GREEN	24D-1	DRILL-5
EoP-1 direct execution	GREEN	frozen P18 + GATE-2	DRILL-12
EoP-2 auto-execution	GREEN	frozen P18	DRILL-2
EoP-3 content injection	GREEN	24C-3	DRILL-3
EoP-4 confused deputy	GREEN	24D-2a	DRILL-6
EoP-5 provenance laundering	GREEN	24C-2	DRILL-4
EoP-6 alternate mutator import	GREEN	24B GATE-2	DRILL-9, DRILL-12
EoP-7 client-supplied effect	GREEN	24C-1	DRILL-7
EoP-8/14 proposal TOCTOU	GREEN (hash + per-client)	24C-2b + 24D-4	DRILL-8a; per-client grant revocation closed by 24D-4 (E-017, I-24D4-1)
EoP-9 token theft → weapon	GREEN	24D-1 + 24D-3	DRILL-5, DRILL-10
EoP-10 queue poisoning	GREEN	24D-3	DRILL-10 (+ bulk-reject)
EoP-11 presentation collapse	GREEN (voice + data-layer)	24C-3	DRILL-3 — <i>visual UI capstone = cross-phase, encoded in render_hardening</i>
EoP-12 direct DB/event write	GREEN (gateway unreachable)	24B GATE-2	DRILL-8c, DRILL-9
EoP-13 scope drift	GREEN	24D-2a	DRILL-6 (+ completeness I-24D2a-7)
EoP-15 aux laundering	GREEN (data-layer)	24C-3	DRILL-3
ID-0 default-never	GREEN	24B + completeness	DRILL-1
ID-1 queue-status redaction	GREEN	24B-2	DRILL-1/6 + queue suite
ID-2 per-client read scope	GREEN	24D-2a	DRILL-6
ID-3 polling/read-flood	GREEN	24B-2 + 24D-3	DRILL-10 + cadence suite
ID-4 static view-model	GREEN	24B	server.test I-24B1-6
ID-5 uniform denial	GREEN	24B	DRILL-11
Spoofing	GREEN	24D-1	DRILL-5

Control	v3 status	Built	Drilled (24E)
Tampering	GREEN	24C-2/2b	DRILL-7, DRILL-8
Repudiation	GREEN	24C-2 + 24D-1	DRILL-4 (+ connection audit)
Denial of Service	GREEN	24D-3	DRILL-10
stdio-local-≠-trusted	GREEN	24D-1	DRILL-5

The core proof (EoP-1/EoP-6): `runtime.runTool` call-site count is **EXACTLY 2** with the whole gateway built — both in the frozen `chat/` tree, none in the gateway (DRILL-12). The gateway added ZERO mutation paths.

The five close-gates — final status

- **GATE-1** (FC-2 wraps Phase 18 additively): **PASS** (256048c , Verdict A).
- **GATE-2** (transitive import allowlist): **GREEN** (24B; DRILL-9/DRILL-12).
- **GATE-3** (canonical effect un-forgable): **GREEN** (24C-1; DRILL-7).
- **GATE-4** (human-session authority marker): **NOT a gateway control / not built**. The gateway has NO path to emit an approval-decision event — it cannot reach the executor or any lifecycle/event writer (GATE-2; DRILL-8c). The marker would defend the *approval UI's* own decision emission, a surface the gateway never touches; no drilled gateway control relies on it. It remains a design item for the approval-UI layer, **outside the Phase-24 gateway freeze**, and is honestly NOT claimed as built.
- **GATE-5** (enqueue needle's eye): **GREEN** (24C-2; DRILL-7 rejects every client-authored field at the boundary).

Residuals — none open

- **E-017 (CLOSED by 24D-4)**: per-client GRANT revocation IS now re-checked at the decision point. The approval row persists the gateway's `client_id` (additive nullable column, no new migration id); the guard re-reads that client's CURRENT grant via the shared canonical-policy leaf (`clientGrantStillAuthorizes`) + an injected `CurrentGrantLookup` , and DENIES (`revalidation_grant_revoked`) before `runtime.runTool` if the grant no longer authorizes the capability. Drilled: I-24D4-1 (revoked → DENIED, `runTool` not called), I-24D4-2 (still-granted → executes once). Legacy approvals unchanged; sole executor intact (count 2). Phase 24 has ZERO open residuals. (E-018 is an APPLIED test-config item, not a threat.)

Cross-phase constraints still standing

- **EoP-11 visual UI capstone**: the trusted/untrusted channel separation is GREEN in voice + data (DRILL-3); the future UI must honor the encoded `render_hardening` requirement (no channel collapse in pixels).
- **EoP-13 + ID-0 standing completeness tests** remain build-failing gates (new mutating capability → non-exposable by default; new readable surface → NEVER-EXPOSED by default).

Phase 24 gateway: FROZEN 2026-06-17. Closeout: `PHASE24_CLOSEOUT.md` .